

| PROCESSES — fork/exec/wait

fork()

- **0 in child, child pid in parent** - 1 err (errno=EAGAIN if proc limit).
- Child = copy of parent VAS (COW pages until write).
- Inherits: FDs (offsets **shared()**), sig handlers, env, cwd, pgid, umask, real/eff UID, resource limits.
- NOT inherited: pid, ppid, file descriptors, pending signals, CPU times, alarms.
- After fork: only async-signal-safe fns before exec in multi-threaded program.

exec*() family

- Replaces VAS w/ new program. PID/PPID stay same.
- **FDs persist** (unless O_CLOEXEC / fcntl FD_CLOEXEC).
- Handlers → **SIG_DFL, SIG_IGN persists**. Mask preserved.
- execvp searches PATH; execv needs full path; execlt takes envp.
- l-variants: list of args. v-variants: array. p: PATH search. e: env passed.
- Returns only on failure (-1, errno set).

wait / waitpid

- wait(&st): block until ANY child changes state.
- waitpid(pid,&st,&opt): pid=-1 any; pid=0 same pgpr; pid>0 specific; pid<-1 pgpr [pid].
- **WNOHANG**: 0 if running, pid if state change, -1 (ECHILD) no children.
- **WUNTRACED**: catch stopped; **WCONTINUED**: catch resumed.
- WIFEXITED+WEXITSTATUS / WIFSIGNALED+WTERMSIG+WCOREDUMP / WIFSTOPPED+WSTOPSIG / WIFCONTINUED.

Zombie: exited, unreaped — entry in process table consuming a PID slot. **Orphan**: parent died → reparented to PID 1 (init), reaped automatically. SIGCHLD handler should reap to avoid zombies.

Busy-wait vs Block

- while(waitpid(-1,&st,WNOHANG)==0) /*spin*/;
- Returns 0 = child alive, no state change, -1 wastes CPU.
- Hangs forever: kill(pid,STOERM), then SIGKILL.
- Block: omit WNOHANG, or use SIGCHLD+sigsuspend.

fork() output orderings — classic Q

Parent prints "waiting", waits; child sleeps 5, prints "child done"; parent prints "done".

- First line could be: "Parent waiting..." OR "Child done" (race).
- NEVER first: "Parent done" (gated by wait).
- Remove wait(): parent may exit before child → child becomes orphan, reparented to init which reaps later.

Cost: fork vs fork+exec vs pthread

Mode	Cost
fork	New PCB, PT copy w/ COW. Pages shared until write. Cheap if no writes.
fork+exec	+ tear down old VAS, build new, load image, cold TLB+cache .
pthread	No new VAS — just stack + thread struct. Cheapest. 1-core: ctx switch+shared cache may thrash.

Process states

State	Meaning
R	Running/Runnable
S	Sleeping (interruptible)
D	Uninterruptible sleep (I/O — can't even kill)
T	Stopped (SIGSTOP/SIGTSTP)
Z	Zombie (exited, unreaped)
X	Dead

| FILE DESCRIPTORS & PIPES

FD model — 3 levels

- **Per-proc FD table**: index → open-file entry.
- **System open-file table**: offset, flags, mode. Shared across fork+dup.
- **dup/vnfd table**: actual file.
- 0=stdin, 1=stdout, 2=stderr. open returns lowest unused FD.
- **dup(fd)**: lowest unused FD aliasing fd. **dup2(o,n)**: closes n if open, makes n alias o. **dup3()**: like dup2 + flags.
- fcntl(fd,F_SETFD,FD_CLOEXEC): close on exec.
- O_NONBLOCK: read/write don't block; return EAGAIN.
- O_APPEND: writes always at EOF (atomic in kernel).

pipe()

- int p[2]; pipe(p); // [0]=read [1]=write
- ~64 KB kernel buffer. Byte stream, one-way.
- **read empty pipe BLOCKS** until data OR **all write ends** closed → EOF (0).
- **write no readers** → SIGPIPE (default: term) + EPIPE.
- Forget to close unused ends → reader hangs forever waiting for EOF.
- Atomic write ≤ **PIPE_BUF** (4 KB).
- Pipes are FIFO, no seek. Named pipes (FIFOs): mkfifo.

Need waitpid before read? NO. read blocks until write or EOF. waitpid only reaps zombie — orthogonal to data flow.

Counting child FDs (classic Q)

Child did: close(p[0]); dup2(p[1], STDOUT); before execvp.

FD	Refers to
0	stdin → terminal
1	STDOUT → pipe write end
2	stderr → terminal
3	orig p[1] — still open (dup2 doesn't close src)

Total = 4 FDs. Always count: 0,1,2 + every still-open FD.

Pipe in shell: ls | wc

```
int p[2]; pipe(p);
if(fork()==0) { dup2(p[1],1); close(p[0]);
  close(p[1]); execvp("ls*...",); }
if(fork()==0) { dup2(p[0],0); close(p[1]);
  close(p[0]); execvp("wc*...",); }
close(p[0]); close(p[1]); // PARENT must close!
waitpid(-1,0,0); waitpid(-1,0,0);
```

3-stage pipeline: a | b | c

```
int p1[2],p2[2]; pipe(p1); pipe(p2);
// a: stdout=p1[1]
// b: stdin=p1[0], stdout=p2[1]
// c: stdin=p2[0]
// parent: close ALL pipe FDs
```

open() flags reference

Flag	Effect
O_RDONLY WRONLY RDWR	Access mode
O_CREAT	Create if missing (need mode arg)
O_EXCL	With CREAT: fail if exists (atomic)
O_TRUNC	Truncate to 0 if exists
O_APPEND	Atomic append
O_NONBLOCK	Non-blocking I/O
O_CLOEXEC	Auto-close on exec

File offset sharing

- fork()/dup() → child and parent share offset (same open-file entry).
- Two open() of same file → independent offsets.
- Race: 2 procs writing same file → use O_APPEND for atomicity.

| SIGNALS

API table

Fn	What
sigaction(s,&sa,old)	Install handler/disposition
signal(s,h)	Legacy, semantics vary, avoid
sigprocmask(how,&so)	SIG_BLOCK / UNBLOCK / SETMASK
pthread_sigmask	Per-thread mask
sigpending(&si)	Get pending sigs
sigsuspend(&m)	Atomic mask + sleep until caught sig
pause()	Sleep until any sig — has TOCTOU race
kill(pid,s)	Send: pid<-1→pgpr[pid]; pid=0 own pgpr; pid=-1 all
raise(s)	Send to self
alarm(n)	SIGALRM arrives in sec
settimer	Periodic alarms
sigempty/fill/add/delset	Build sigset t
sigismember(&sa,sig)	Test membership

3 Dispositions

- **SIG_DFL**: default action (term / ignore / stop / core depending on sig).
- **SIG_IGN**: signal discarded silently; pending ones dropped too.
- **Custom handler: caught** — sigsuspend returns AFTER it runs.

sigsuspend pitfall: If disposition is SIG_IGN, signal dropped → sigsuspend NEVER returns. Must install REAL handler (even no-op) for it to wake.

Correct "wait for SIGINT" pattern

```
sigset_t b, e;
sigemptyset(&b); sigaddset(&b, SIGINT);
sigprocmask(SIG_BLOCK,&b,NULL); // 1. BLOCK
struct sigaction sa={.sa_handler=nop};
sigemptyset(&sa.sa_mask);
sigaction(SIGINT,&sa,NULL); // 2. real handler
sigemptyset(&e); // 3. atomic unblock+wait
sigsuspend(&e);
```

Why order matters

Install handler BEFORE first chance signal could arrive. If sigprocmask done AFTER sigaction but signal arrives before BOTH — default action runs (may kill program). Akash's claim "order doesn't matter" is **WRONG**.

Signals are async — kernel-driven

Handlers NOT called by your C code. Kernel interrupts execution at any instruction boundary, saves context, runs handler with own mask, restores. Implication: handlers must use only async-signal-safe fns (write yes, printf/malloc NO). Use only volatile sig_atomic_t for flag variables touched in handler.

Pending sigs

- Sig sent while blocked → **pending**.
- Standard sigs: **1 pending max** (don't queue). Realtime sigs (SIGRTMIN..MAX) DO queue.
- SIG_IGN'd sigs never become pending — dropped.
- Unlocking delivers pending → handler runs.
- fork() clears child's pending sigs.

Common signals

Sig	Default	Trigger
SIGINT	term	Ctrl+C
SIGQUIT	core	Ctrl+Q
SIGTERM	term	polite kill (default)
SIGKILL	term	uncatchable (#9)
SIGSTOP	stop	uncatchable
SIGTSTP	stop	Ctrl-Z
SIGCONT	stop	resume stopped
SIGCHLD	ignore	child state change
SIGTTIN	stop	bg reads terminal
SIGTTOU	stop	bg writes/tcsetpgpr
SIGPIPE	term	write to closed pipe
SIGSEGV	core	invalid memory
SIGBUS	core	bad alignment
SIGFPE	core	arithmetic err.
SIGILL	core	illegal instruction
SIGALRM	term	alarm() timer
SIGUSR1/2	term	user-defined
SIGHUP	term	terminal hangup
SIGABRT	core	abort()

SA_RESTART — slow vs fast

- **Slow devices** (pipes, sockets, terminals): syscalls interruptible → fail w/ EINTR unless SA_RESTART. WHY: may block indefinitely; signal lets process cancel/timeout — useful escape.
- **Fast devices** (disk, FS): NOT interruptible. WHY: expected fast; interruption adds restart logic + partial-completion mess → inefficient overhead with no real upside.

sa flags reference

Flag	Effect
SA_RESTART	Auto-restart slow syscalls
SA_NOCLDSTOP	No SIGCHLD on stop
SA_NOCLDWAIT	Auto-reap children (no zombies)
SA_NODEFER	Don't block self in handler
SA_RESETHAND	Reset to DFL after one delivery
SA_SIGINFO	Use sa_sigaction (extended info)
SA_ONSTACK	Run handler on alt stack

sigsuspend race-free SIGCHLD wait

```
sigset_t b, e, old;
sigemptyset(&b); sigaddset(&b, SIGCHLD);
sigprocmask(SIG_BLOCK,&b,&old);
struct sigaction sa={.sa_handler=on_chld};
sigaction(SIGCHLD,&sa,NULL);
while(!done) { sigemptyset(&e);
  sigsuspend(&e); }
sigprocmask(SIG_SETMASK,&old,NULL);
```

Sig handler safety rules

- Only async-signal-safe fns: write, _exit, read, kill, sigprocmask, time, sem_post.
- NOT safe: printf, malloc, free, fprintf, anything with locks.
- Set volatile sig_atomic_t flag; let main loop see it.
- Or use self-pipe trick: handler writes 1 byte to pipe; main reads.
- SA_SIGINFO gives signo, t (sender pid, code, value).

Signal vs syscall interaction

- Signal during blocking sysca on slow dev: EINTR (unless SA_RESTART).
- Always check for EINTR and retry if needed: do { r=read(...); } while(r== -1 && errno==EINTR);
- Handlers preempt main code → guard shared state.

| PROCESS GROUPS & TERMINAL

Hierarchy

Session ⇒ process groups (pgpr) ⇒ processes. Session has one controlling terminal. ONE pgpr is **foreground**; others bg.

Call	Effect
setpgid(pid,pgid)	Move pid to pgpr pgid
getpgpr(i) / getpgid(pid)	Get pgpr
getsid(pid)	Get session ID
setsid()	New session+pgpr (not if already pg leader)
tcsetpgpr(fd,pgid)	Set fg pgpr on terminal
tcgetpgpr(fd)	Get fg pgpr

FG vs BG

- **FG pgpr**: receives terminal input, gets SIGINT on Ctrl+C, SIGTSTP on Ctrl+Z, SIGQUIT on Ctrl+V.
- **BG pgpr reading terminal** → SIGTTIN (stop).
- **BG pgpr writing terminal** (if TOSTOP set) → SIGTTOU.
- **BG calling tcsetpgpr** → SIGTTOU always.
- FG pgpr on terminal hangup → SIGHUP.

Is tcsetpgpr(STDIN, getpgpr()) needed in fg child?

NO. Forked child inherits parent's pgpr. Parent already fg → child also fg. Call is redundant but harmless. ONLY needed when moving child into a NEW pgpr (shell job control).

Block SIGTTOU if tcsetpgpr moves to child?

No need. SIGTTOU only sent when caller is in a BG pgpr. Both parent & child are fg → no SIGTTOU. Block SIGTTOU only if caller may be backgrounded.

Shell foreground-job template

```
pid = fork();
if(pid==0) { // child
  setpgid(0,0); // own pgpr (race-safe dup)
  signal(SIGTTOU,SIG_DFL); // reset
  tcsetpgpr(STDIN,getpgpr());
  /* reset SIGINT/QUIT/TERM to DFL */
  execvp(argv[0],argv);
}
setpgid(pid,pid); // race-safe dup
tcsetpgpr(STDIN,pid);
waitpid(pid,&st,WUNTRACED);
tcsetpgpr(STDIN,getpgpr()); // shell reclaims
```

Background job template

```
pid=fork();
if(pid==0) { setpgid(0,0); execvp(...); }
setpgid(pid,pid);
/* DON'T tcsetpgpr — leave shell in fg */
/* track in job table; reap via SIGCHLD */
```

Sending signal to pgpr

kill(-pgid, sig) sends to whole group. Ctrl+C → kernel sends SIGINT to entire fg pgpr.

| OUTPUT TRACE — pipe redirect

Child: dup2(p[1],STDOUT); execvp("echo","child in control"). Parent reads pipe, writes bytes to STDOUT. Result: **"child in control" appears on terminal**. Why: echo writes to FD 1 (now pipe write end). Parent reads from p[0], writes those bytes to its own FD 1 (terminal). Without dup2, echo prints direct to terminal.

| SHELL GOTCHAS

- After execvp, handlers reset → MUST reset to DFL in child before exec (otherwise inherited SIG_IGN persists, Ctrl+C won't kill child).
- Race: parent sets child's pgpr, but child may exec first. Fix: setpgid in BOTH parent & child.
- tcsetpgpr must be called from a proc in the same session.
- Stop fg job: SIGTSTP delivered. Shell uses WUNTRACED to learn of it. Resume: send SIGCONT to pgpr + tcsetpgpr back.
- If shell exits without reaping bg jobs → init handles them.
- Reading from terminal in bg with SIGTTIN ignored = EIO error.

| FAST KILL DECISIONS

Want to...	Use
Stop child cleanly	SIGTERM
Force kill	SIGKILL
Pause	SIGSTOP
Resume	SIGCONT
Custom event	SIGUSR1/2
Notify child died	SIGCHLD (auto)
Reload config	SIGHUP (daemon convention)
Generate core	SIGQUIT

| EXIT

Call	Effect
_exit(n)	Call atexit, flush stdio, then _exit
_exit(n)	Immediate, no buffer flush
return from main	= _exit(rv)
abort()	Raise SIGABRT → core dump
atexit(fn)	Register cleanup fn (LIFO)

After fork, child should use **_exit** not exit (avoid double flush of buffered stdio).

| STDIO BUFFERING + FORK

```
printf("hi"); // buffered (line/full)
fork(); // both have "hi" in buffer
exit(0); // both flush - "hihi" printed!
Fork: fflush(stdout) before fork OR setbuf(stdout,NULL) .
```

TWO-CHILD PIPE EXAMPLE (Joel's Q)

```
for(i=0;i<3;i++){
  pipe(fds[i]); pid=fork();
  if(pid==0) { close(fds[i][0]);
    write(fds[i][1],&i,sizeof i);
    _exit(0); }
  else { close(fds[i][1]); int r;
    waitpid(pid,0,NULL); // - NOT needed
    read(fds[i][0],&r,sizeof r);
    printf("Child %i\n",r);
  }
}
```

Joel's claim that waitpid is needed to "not miss" data: **WRONG**. read blocks until data or EOF. waitpid only reaps.

| SELF-PIPE TRICK

Make signals interrupt select()/poll() loop:

```
int sp[2]; pipe(sp);
void handler(int s) { write(sp[1],"*",1); }
// main loop: select on sp[0] + other FDs
// On signal, handler writes; select wakes.
```

FILE SYSTEMS**FAT (File Allocation Table)**

- Layout: boot block + FAT + data region.
- FAT[i] = next block index, EOF marker, or 0 (free).
- Dirent: name + start block + size + attrs.
- File = linked list of blocks via FAT chain.
- Simple, recoverable, ubiquitous, no fragmentation issues for small files.
- Linear scan for free blocks; random access slow (walk chain); poor for large files; no permissions in basic FAT.
- Used in: FAT12/16/32, exFAT, SD cards, USB drives.

Inode FS (ext2/3/4)

- Layout: **superblock** | **inode bitmap** | **block bitmap** | **inode table** | **data blocks**.
- Superblock: FS-wide meta (size, block size, free counts, magic).
- Inode: file metadata + block pointers. **Filename NOT in inode**.
- Inode fields: mode (type+perms), uid/gid, size, atime/mtime/ctime, link count, block pointers.
- Block pointers: ~12 direct + 1 single-indirect + 1 double + 1 triple → huge files possible.
- Directory** = file whose data blocks contain **dirents** (name, inode#).
- ext3 adds journaling; ext4 adds extents (range-based).

File size limits (rough)

- 12 direct ptrs × 4 KB block = 48 KB direct.
- + single indirect: 1024 ptrs × 4 KB = 4 MB.
- + double: 1024² × 4 KB ≈ 4 GB.
- + triple: 1024³ × 4 KB ≈ 4 TB.

Path resolution: /home/r/x.txt

- Start at root inode (well-known #2 in ext2).
 - Read root's data block → find "home" → inode#.
 - Read /home inode → find "r" → inode#.
 - Read /home/r inode → find "x.txt" → inode#.
 - Read x.txt inode → data blocks = file content.
- Each step = potentially 1 disk read → why FS caches dentries.

Hard link vs Symlink

	Hard	Symlink
What	Dirent → same inode	File holding a path
Inode	Shared	Own (own meta)
Cross-FS	No	Yes
To dirs	No (cycles)	Yes
Dangling	Never	Possible
Survives rename	Yes	No
Survives target delete	Yes (refcnt>0)	Dangling
chmod	Inode (target too)	Symlink itself (usually)
Size	Same as target	Length of path string
Type (ls -l)	-d/etc	l

When to free inode? Inode keeps **reference (link) count**. Each hard link → ++count. Each unlink → --count. Free inode + data blocks when count=0 AND no open FDs reference it (Linux waits for last close).

Symlink unique capability: independent metadata (own owner, perms, timestamps) since separate inode. Also: span FSes, point to dirs, point to nonexistent path.

Symlink danger: dangling refs → target moved/deleted, symlink points nowhere. Hard links can't dangle.

Deleting fruit/ with 5 files

Inodes: 1+1+5 = 7 (root, fruit, 5 files).

On-disk ops:

- Remove dirent for "fruit" from root's data block.
- For each file: free inode (inode bitmap), free data blocks (block bitmap).
- Free fruit's directory data blocks; free fruit's inode.
- Update superblock free counts; link counts on dirs; timestamps.

Power-loss inconsistencies (no journal)

- Dirent removed but inode/blocks not freed → orphan blocks (space leak).
- Inode freed but dirent remains → dangling reference / corruption.
- Bitmaps say "free" but inode says "alive" → cross-link, future overwrite of live data.
- Block bitmap says free but block actually contains user data → loss on next alloc.
- Fix: **journaling** → log intended ops to journal area, replay on reboot.

Other FS concepts

- fsync(fd)**: flush all buffered file data + metadata to disk.
- fdatasync**: data only, skip metadata if size unchanged.
- mmap'd file**: changes flushed on msync or unmmap.
- Block size** typically 4 KB; matches page size.
- VFS (Linux)**: abstraction over multiple FS types.

THREADS**Threads vs Processes**

Shared (same proc)	Per-thread
Heap, globals, code	Stack
FD table	Registers, PC
Sig handlers (proc-wide)	Sig mask (per-thread)
cwd, umask	errno (thread-local)
Page tables	TLS (thread)
Memory mappings	alt stack

pthread API

```
int pthread_create(pthread_t*, attr,
    void*(*fn)(void*), void*arg);
int pthread_join(pthread_t, void**ret);
int pthread_detach(pthread_t);
void pthread_exit(void*ret);
pthread_t pthread_self(void);
int pthread_cancel(pthread_t);
```

- Detach: cleans up self, can't be joined.
- Don't return ptr to local stack → invalid after thread exits.
- Joining a detached thread = UB.
- pthread_exit in main keeps process alive until all threads done.

Scenario C — 2-thread prime counter

```
int main() {
    pthread_t t1, t2;
    int cA=0, cB=0;
    pthread_create(&t1, NULL, fA, &cA);
    pthread_create(&t2, NULL, fB, &cB);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("%d\n", cA+cB);
    return 0;
}
```

Why threads help (real-time / games)

Different parts make progress independently. If one blocks on hardware, others keep running. **Hides I/O latency**, reduces input lag. Even 1 core: overlapping I/O wait w/ CPU work reduces idle.

- Input thread: blocked on controller.
- Audio: feeding sound buffer.
- Render: prepping next frame.
- Game logic: positions/collisions.

SYNCHRONIZATION**Mutex pattern**

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_lock(&m);
/* critical section */
pthread_mutex_unlock(&m);
// Variants:
pthread_mutex_trylock(&m); // EBUSY if held
pthread_mutex_timedlock(&m, t); // timeout
```

Race condition: ≥2 threads, ≥1 writes shared state, outcome interleaving-dependent. **++ NOT atomic**.

Mutex attributes

- NORMAL**: default; deadlocks if same thread locks twice.
- RECURSIVE**: same thread can lock N times, unlock N times.
- ERRORCHECK**: returns error on relock/wrong unlock.

Condition variables

```
pthread_mutex_lock(&m);
while(!condition)
    pthread_cond_wait(&cv, &m); // atomic unlock+sleep
/* use shared state */
pthread_mutex_unlock(&m);
// Signaler:
pthread_cond_signal(&cv); // wake one
pthread_cond_broadcast(&cv); // wake all
```

- ALWAYS** check condition in a **while** (spurious wakeups).
- Hold mutex when signaling for predictable wakeup ordering.
- cond wait re-acquires mutex before returning.

Semaphores

```
sem_t s; sem_init(&s, 0, N); // N = init count
sem_wait(&s); // -- (block if 0)
sem_post(&s); // ++
```

Use for: bounded buffer (count = empty slots), counting limited resources, signaling.

Disabling preemption / interrupts

- 1 core**: sufficient — no other thread can run, CS atomic.
- Multi-core**: NOT sufficient — other cores still execute concurrently → race on shared kernel data. Need locks/spinlocks.

Locks vs disable preemption on 1 core (Lifeless Locks): If holder preempted mid-CS, another task may acquire-attempt → spins or blocks even though holder isn't running → wasted CPU + scheduling overhead. Disabling preemption guarantees CS completes without ctx switch.

Spinlock vs Mutex

	Spinlock	Mutex
Wait	Busy-spin	Sleep, ctx switch
Uncontended cost	Lower (atomic CAS)	Higher (syscall path)
Waiting cost	CPU cycles	Ctx switch
Best for	Very short CS, multicore	Long CS, blocking I/O
1 core	Wasteful — spinner blocks holder	Sleep lets holder run
Kernel	Yes (IRQ off context)	Process context only

FS block locks? CS includes write+flush = long, blocking I/O → **mutex**.

Deadlock — 4 conditions

- Mutual exclusion.
- Hold-and-wait.
- No preemption.
- Circular wait.

Break any one to prevent.

Lock-all-first CAN deadlock if different orders.

T1 wants [1,2]; locks 1, waits 2.

T2 wants [2,1]; locks 2, waits 1.

Circular wait. Fix: global ordering rule (always ascending block #).

Other deadlock strategies

- Try-lock w/ backoff**: can't get all → release all, retry.
- Lock hierarchy / ranks**.
- Detection + recovery**: wait-for graph, kill victim.
- Avoidance**: Banker's algorithm (needs claim).
- Two-phase locking**: acquire phase, release phase.

Why lock multiple blocks atomically (FS)?

Grouped updates (dirent + inode + bitmaps) must be consistent. Partial visibility ↔ another thread races on same group → corruption or torn read.

Classic sync problems

- Producer/consumer**: bounded buffer, mutex + 2 CVs (not-full, not-empty), or 2 semaphores.
- Readers/writers**: many readers OR one writer. Beware writer starvation.
- Dining philosophers**: 5 forks, deadlock if all grab left → break symmetry, or pick up both atomically, or resource hierarchy.
- Sleeping barber**: sema for waiting customers + barber.

Atomic operations

- CAS (compare-and-swap): primitive for lock-free algorithms.
- C11: `Atomic_int, atomic_fetch_add`.
- Memory barriers / fences enforce ordering on weakly-ordered CPUs.

SCHEDULING

Algo	How	+	–
FCFS	FIFO	Simple	Convoy effect
SJF	Shortest next (non-preempt)	Min avg wait	Starvation; needs estimate
SRTF	Preempt for shorter remaining	Responsive	Starves long jobs
RR	Quantum, rotate	No starvation; equal opp	No reward for yielders
Priority	Highest first	Important first	Starvation w/o aging
CFS	Min runtime (RB-tree); min time-slice	Equal CPU; rewards I/O	Complex
MLFQ	Multiple queues; demote hoqs, promote I/O	Adapts	Many tunables
Lottery	Random tickets weighted by share	Probabilistic fairness	Variance

SRTF Pros: responsive to short/interactive tasks; better throughput; fewer ctx switches if first task is shortest. **SRTF Cons:** starves long jobs — if shorter jobs keep arriving, long ones never run.

RR Pros: fair, no starvation, equal opportunities. **Cons:** ignores I/O-bound yielding; quantum too small → ctx-switch overhead, too big → unresponsive.

CFS vs RR — when differ?

When tasks **yield early** (I/O-bound), CFS sees they used less CPU (lower vruntime) → prioritizes them when ready. RR just gives equal turn later. CFS handles I/O-bound + CPU-bound mixes better.

MLFQ vs RR — mixed workload

Short network/I/O handlers → stay in high-priority short-quantum queue → low response time. CPU-bound → demoted to long-quantum low-priority. RR treats all equally → interactive starves alongside CPU hogs.

SRTF worked example

A(arr 0, run 6), B(2,3), C(6,2), D*(arr 1, run 1, off 3 cycle).

t	Run	A	B	C	D*
0	A	6→5	—	—	—
1	D*	5	—	—	1→0
2-4	B	5	3→0	—	off
5	D*	5	0	—	1→0
6-7	C	5	0	2→0	off
8	A	5→4	0	0	off
9	D*	4	0	0	1→0
10-13	A	4→0	0	0	off
14	D*	0	0	0	1→0

LIFO scheduler subtlety

- LIFO non-preempt**: first prog keeps CPU until block/yield → installs handler + sigsuspend before sender runs → safer.
- LIFO + preempt**: sender preempts immediately on start → may send sig BEFORE handler installed → default kills program.

Scheduling metrics

- Throughput**: jobs/sec.
- Turnaround**: completion — arrival.
- Wait time**: time in ready queue.
- Response time**: first run — arrival.
- CPU utilization**: % busy.

Context switch cost

- Save/restore registers, switch page tables, cold TLB-cache.
- ~1-10 μs direct, more from cache pollution.
- Goal: minimize unnecessary switches.

I/O-bound vs CPU-bound

- I/O-bound: short CPU bursts, frequent blocking. Want quick scheduling on wake.
- CPU-bound: long CPU bursts, rarely blocks. OK with long quanta, lower priority.
- Most workloads = mix → adaptive schedulers (CFS, MLFQ) win.

P3: LOCALITY · BUFFERING · VIRTUAL MEMORY · MEMORY ALLOCATION

MEMORY HIERARCHY

Level	Size	Latency
Register	~1 KB	0.3 ns
L1 cache	32-64 KB	1 ns
L2 cache	256 KB-1 MB	3 ns
L3 cache	4-32 MB	10 ns
RAM	GB	100 ns
HDD	TB	50 μs
SDD	TB	5 ms
Network	—	1-100 ms

Locality types

- Temporal: recently used → likely used again. (loop vars, hot data)
- Spatial: nearby addresses likely accessed soon → caches load whole **cache lines** (typ. 64 B).

Cache basics

- Hit: fast. Miss: load from next level → stall.
- Miss types: **compulsory** (cold), **capacity** (too much data), **conflict** (mapping collision).
- Eviction: LRU, random, pseudo-LRU.
- Write-through vs write-back; write-allocate vs no-write-allocate.
- HW prefetcher may grab next sequential line on miss.
- Direct-mapped vs set-associative vs fully-associative.

Cache line organization

addr = [tag | set index | block offset]. Set assoc N → N ways per set. Larger blocks → more spatial locality but more conflict misses.

AoS vs SoA (BST Q)

- Key 4B + Value 4B; cache line 64B.
- AoS (struct {k,v}): 8B/node → **8 nodes/line**. First 3 BST layers (7 nodes) → **1 miss** grabs all 7 + values.
- SoA (sep arrays): 16 keys/line, values elsewhere → **2 misses** (1 key, 1 value).
- Shallow searches: **AoS wins**.

Deep-tree prefetch: on miss at index i, fetch **children at 2i+1, 2i+2 (+ grandchildren)** instead of next sequential line.

False sharing

Two threads write different vars in same cache line → line bounces between cores via coherence protocol → severe slowdown despite no real conflict. Fix: padding to 64B, alignas.

Loop optimizations for cache

- Loop interchange: traverse row-major arrays row-by-row, not column.
- Blocking/tiling: process data in cache-sized chunks.
- Data layout: hot fields together; AoS vs SoA matters.

BUFFERING

- Amortize slow syscall/device cost by batching ops.
- stdio: terminal=line-buffered, file=fully-buffered, stderr=unbuffered.
- fflush() forces; setvbuf changes mode.
- Pipe kernel buf ~64 KB. write blocks if full.
- Flush before fork — child inherits buffered content → duplicate output buf.
- Kernel page cache buffers disk I/O.
- setvbuf(stdout, NULL) = unbuffered (slow but immediate).

Why buffer? Cost analysis

1 syscall ~1 μs. Writing 1M bytes 1-at-a-time = 1M syscalls = 1 sec. Buffer 4KB at a time = 250 syscalls = ~250 μs. **~4000x faster**.

VIRTUAL MEMORY — Why

- Isolation: proc A can't read/write proc B's memory.
- Abstraction: contiguous VA over fragmented PA.
- Demand paging: only resident pages need frames; lazy.
- Sharing: read-only code/libs across procs.
- Protection: per-page R/W/X bits, user/kernel.
- Swap: back pages with disk when RAM full.
- Overcommit: VA > physical RAM possible.

ADDRESS TRANSLATION

VA = [PT indices] [offset]. Offset bits = log2(page_size). Frame# from PTE; offset preserved → PA.

Example 32-bit, [4 | 10 | 18]

- Page size = 2¹⁸ = **256 KB**.
- Top-PT entries = 2⁴ = 16.
- L2-PT entries = 2¹⁰ = 1024.
- Virtual pages = 2⁴(4+10) = **16,384**.
- If all populated: 16 L2 tables × 1024 entries.

Worked: 0b 1000 | 0000000001 | 000000000000011111

- Top idx = 0b1000 = **8**.
- L2 idx = 0b0000000001 = **1**.
- Offset = 0b...011111 = **31**.
- top-PT[8] → addr of L2-PT. L2-PT[1] → PTE → frame F.
- PA = (F << 18) | 31.

x86-64 4-level PT (real-world)

48-bit VA: [9 | 9 | 9 | 9 | 12] = PML4 / PDPT / PD / PT / offset. 4 KB pages. Each level 512 entries × 8B = 4KB.

Walking = 4 memory loads on TLB miss.

PTE bits (typical)

Bit	Use
P (Present)	0 → page fault
Frame #	Phys frame
R/W	Write perm
U/S	User or kernel only
A (Accessed)	Set on any access (for LRU approx)
D (Dirty)	Set on write (for writeback)
NX	No-execute
PCD/PWT	Caching mode
G	Global (don't flush on ctx switch)

Smaller pages (e.g., 12-bit → 4 KB)

- + Less **internal fragmentation**.
- + Finer protection & sharing granularity.
- + Less wasted RAM if app uses small chunks.
- Bigger PTs (more PTEs).
- More TLB pressure (each entry covers less).
- More page faults per byte.

64-bit VAs — why not terrifying

PTs allocated **lazily**. OS only creates lower-level PT pages for VA ranges actually used. PT memory × **actively mapped VA**, not 2⁶⁴. Most of the 64-bit space is just "not mapped, no PT needed".

Process VAS layout (typical Linux)

Region	Notes
text (code)	RX, shared
rodata	R only
data	RW initialized globals
bss	RW zero-init
heap ↑	brk grows up
mmap region	shared libs, big allocs
stack ↓	grows down
kernel space	top, U-bit unset

TLB

- HW cache of recent VA→PA translations (PTEs). 64-1024 entries typ., fully/set-associative.
- Hit: ~1 cycle. Miss: walk PT (~100+ cycles, multiple memory accesses).
- Ctx switch: flush TLB (or tag w/ ASID/PCID to skip flush).
- TLB miss ≠ data cache miss — separate structures.
- Separate I-TLB and D-TLB usually.
- 2-level TLBs: small fast L1, larger L2.

HUGE PAGES (2 MB, 1 GB)

What MMU hardware benefits? The **TLB**. Each entry now covers more physical memory (2 MB vs 4 KB) → fewer entries needed for same data → **fewer TLB misses**. Also **shorter page-table walks** — stop at higher level, fewer levels of indirection.

- Need contiguous physical memory.
- Internal fragmentation if app uses only part.
- + Big win for large-footprint apps (DBs, JVMs).
- Linux: explicit hugetlbfs or Transparent Huge Pages.

Transparent Huge Pages worker

Kernel thread periodically **compacts/relocates** in-use phys pages to create contiguous free regions → enables huge-page promotion. Active during/after small alloc/free bursts because **external fragmentation** leaves scattered "holes" needing consolidation.

PAGE FAULTS

Handler flow

- CPU traps to kernel (PTE invalid / perms wrong).
- Kernel checks: in any VMA?
- NO → **SIGSEGV** (segfault).
- Lazy/file-mapped not loaded: pick free frame (or evict), zero-fill or read from disk, update PTE, restart instr.
- Swapped out: read from swap.
- COW (write to RO shared): allocate, copy, mark writable.
- Stork growth: extend VMA + allocate.

Minor fault: page in RAM, just unmapped (fast). **Major fault:** need disk I/O (slow).

Replacement policies

Policy	Idea	Notes
FIFO	Oldest in evict	Belady's anomaly: more frames → more faults possible!
LRU	Least-recently-used	Good, but tracking exact LRU expensive
Clock	Approx LRU: scan ref bits, clear, advance	Cheap; widely used
NRU	Use ref+dirty: 4 classes	Cheap
OPT	Evict whatever used furthest in future	Unimplementable; theoretical bound
LFU	Least frequently used	Can stick on stale popular pages
2nd-chance	FIFO + ref bit reprieve	Compromise

Working set & thrashing

- Working set:** pages actively used in time window. Resident WS ≥ actual WS → low fault rate.
- Thrashing:** too many procs, resident < WS → constant paging, CPU ~0%, disk pinned.
- Fix: swap out whole proc; reduce multiprogramming degree.

COPY-ON-WRITE

- After fork(), parent & child share phys frames marked **read-only** in both PTs.
- On write: fault → kernel allocates new frame, copies, marks writable in writer's PT.
- Avoids huge upfront copy. Esp efficient when child exec()s right away.
- Same idea: shared libs, mmap PRIVATE.
- Tracking: refcount per page; ==1 means safe to write in place.

USER MALLOC

Heap mechanics

- Grows via brk/brkr (small) or mmap (large/threaded).
- Block: **header** (size+alloc bit) + **payload** + opt **footer** (boundary tag).
- Free lists: implicit (scan all), explicit (linked), segregated (bin per size).
- Alignment: typ 8 or 16-byte.

Strategies

Strategy	Idea	Trade-off
First-fit	First big-enough	Fast; head-frag
Next-fit	Resume last spot	Spreads frag
Best-fit	Smallest fitting	Less waste; full scan
Worst-fit	Largest block	Usable leftovers; bad in practice
Segregated	Bins by size	Fast lookup; complex
Buddy	Split powers of 2	Fast coalesce; ≤50% int frag
Slab	Pre-alloc'd same-size obj caches	Kernel inode/dentry caches

Fragmentation

- Internal:** allocated more than asked (rounded up). Waste inside block.
- External:** total free ≥ request but no chunk fits. Scattered holes.

Coalescing

- On free, check neighbors via boundary tags. If free, merge into bigger block.
- Boundary tag (footer size) → **O(1) backward coalesce**.
- Without coalescing, external frag balloons over time.

Memory bugs

- Leak:** alloc never freed.
- Use-after-free:** access freed memory — may seem OK, UB.
- Double-free:** can corrupt allocator state; exploit vector.
- Buffer overflow:** write past block end; corrupts neighbors.
- Tools: Valgrind, AddressSanitizer, MemorySanitizer, hardened allocs.

STACK vs HEAP

	Stack	Heap
Allocation	LIFO, automatic	malloc/free
Size	Fixed (rlimit)	Grows w/ brk/mmap
Cost	~1 instr	Allocator overhead
Lifetime	Until fn returns	Until free()
Concurrency	Per-thread	Shared, needs lock
Pitfalls	Overflow, return ptr to local	Leak, UAF, DF

MMAP

- mmap(addr, len, prot, flags, fd, off);
- Maps file/anon region into VAS.
- MAP_PRIVATE:** COW; writes invisible to others.
- MAP_SHARED:** writes visible to all mappers; flushed to file.
- MAP_ANONYMOUS:** not file-backed (zero pages).
- prot: PROT_READ/WRITE/EXEC/NONE.
- Used by: malloc large allocs, dynamic loader (.so), shared memory IPC.
- Returns MAP_FAILED ((void*)-1) on error, NOT NULL.
- msync flushes shared mappings; munmap releases.

SHARED MEMORY IPC

- POSIX: shm_open + mmap. SysV: shmget+shmat.
- Fastest IPC — no copy.
- Need synchronization (mutex in shared region or semaphore).

JVM TERM MINI-GLOSSARY

- Page:** fixed-size chunk of VA.
- Frame:** fixed-size chunk of PA.
- PTE:** page table entry (frame# + flags).
- VMA:** virtual memory area (region of VA w/ same perms).
- MMU:** hw that translates VA→PA.
- Resident set:** pages currently in RAM.
- Anonymous page:** not backed by file (heap/stack).
- Dirty page:** modified since last sync.

SYSCALL CHEAT TABLE

	Returns / notes
fork()	0 child / pid parent / -1
execvp(file,argv)	only returns on err
_exit(n)	no return; no flush
exit(n)	atexit + flush + _exit
wait(&st)	pid / -1
waitpid(pid,&st,0)	pid / 0 (WNOHANG) / -1
pipe(int p[2])	0 / -1
dup(fd)	new lowest FD
dup2(old,new)	new / -1; closes new
close(fd)	0 / -1
read(fd,b,n)	bytes / 0 (EOF) / -1
write(fd,b,n)	bytes / -1 (EPIPE)
open(p,fl,m)	fd / -1
lseek(fd,off,wh)	new off / -1
fstat(fd,&st)	0 / -1
fcntl(fd,cmd,_)	varies
kill(pid,sig)	0 / -1
raise(sig)	0 / nonzero
sigaction(s,&n,&o)	0 / -1
sigprocmask(h,&s,&o)	0 / -1
sigsuspend(&m)	-1, EINTR
sigempty/addset	0 / -1
setpgid(pid,pgid)	0 / -1
getpgpr()	pgid
tcsetpgrp(fd,pgid)	0/-1 (SIGTTOU)
togetpgrp(fd)	pgid / -1
setsid()	sid / -1
pthread_create	0 / errno
pthread_join	0 / errno
pthread_mutex_lock	0 / errno
pthread_cond_wait	0 / errno
mmap(...)	ptr / MAP_FAILED
munmap(addr,len)	0 / -1
brk/sbrk	new break / -1

STRUCT sigaction

```
struct sigaction {
    void (*sa_handler) (int); // or SIG_DFL/IGN
    sigset_t sa_mask; // blocked DURING handler
    int sa_flags; // SA_RESTART, etc.
    void (*sa_sigaction) (int,siginfo_t*,void*);
};
```

ANSWER TEMPLATES

"Why didn't sigsuspend return?"

Disposition was **SIG_IGN**, sigsuspend wakes only when a **caught handler** runs. SIG_IGN'd signals are silently dropped. **Fix:** install a real handler (even no-op)

"What's wrong with this signal code?" — checklist

(1) SIG_IGN where handler needed for sigsuspend; (2) sigprocmask SIG_UNBLOCK when meant BLOCK; (3) handler installed AFTER block (race); (4) printf/malloc in handler (unsafe); (5) non-volatile non-sig_atomic_t shared var; (6) checking flag without atomicity guarantee.

"Why block SIGINT before sigsuspend?"

Avoid race where SIGINT arrives between check and pause. sigsuspend **atomically** unblocks+waits — no signal can slip through.

"What happens to blocked signal?"

Becomes **pending** in kernel. Standard sigs don't queue (1 max). Unblock → delivered → handler runs.

"Does order of sigaction vs sigprocmask matter?"

YES, if signal can arrive before handler installed, default action applies (may kill). Install handler first, then block, then sigsuspend.

"Need waitpid before read?"

No. read on empty pipe blocks until write or EOF. waitpid only reaps zombie.

"Does tcsetpgrp matter here?"

If parent already fg, child inherits pgrp → child also fg → tcsetpgrp redundant. Only needed when moving child to NEW pgrp.

"Block SIGTTOU in fg child for tcsetpgrp?"

No. SIGTTOU only sent if caller is bg. Fg child has permission already.

"Why does lock-everything-first deadlock?"

Different acquisition orders → circular wait. **Fix: global ordering** (always ascending block #).

"Disable preemption insufficient on multicore?"

Only stops current core. Others still run concurrently → race. Need locks.

"Why locks less efficient than disable preemption (1-core)?"

Holder may be preempted mid-CS; another task tries lock → spins/blocks while holder isn't running → wasted CPU + scheduling overhead.

"Spinlock vs mutex for FS block locks?"

Mutex. CS includes write+flush — long, potentially blocking. Spinlock burns CPU; on 1-core, spinner blocks holder from being rescheduled.

"RR vs MLFQ for mixed workload?"

MLFQ. Quick network/I/O blocks frequently → kept high-priority short-quantum → low response. CPU-bound demoted to long-quantum low-priority. RR gives all equal → interactive starves.

"What MMU structure helps with huge pages?"

TLB. Each entry covers more memory → fewer misses. Plus shorter PT walks.

"Why kernel worker active after small alloc/free?"

External fragmentation. Scattered holes in phys mem. Worker compacts/relocates → contiguous regions for huge-page promotion.

"Why inode refcount?"

Hard links share inode. Free only when refcount=0 AND no open FDs.

"What can symlink do that hard link can't?"

Independent metadata (own owner, perms, timestamps). Cross-FS. Point to dirs. Point to nonexistent path.

"Symlink danger?"

Dangling reference — target moved/deleted, path points nowhere.

"Why 64-bit VAs not terrifying?"

PTs allocated lazily — only for actively-mapped VA. PT memory ∝ mapped VA, not 2⁶⁴.

"Pros/Cons SRTF vs RR?"

SRTF: + responsive, low avg wait, fewer ctx switches. — starvation. RR: + no starvation, equal opportunities. — ignores yielders.

"Threads improve game perf beyond parallelism?"

Other threads run while one blocks on hardware. Hides I/O latency, reduces input lag.

"Diff Scenario A (fork) vs B (fork+exec)?"

A: child shares parent VAS via COW; lazy copy on write. B: child VAS torn down, replaced with new program; cold cache/TLB; higher cost.

"Why use threads instead of multiple processes?"

Cheaper (no PT copy, shared memory). Easy data sharing. But: shared state → need synchronization, one crash kills all threads.

"Why Journaling FS?"

Atomic multi-block updates. On crash, replay log → consistent state. Avoids fsck full scan.

TINY CODE PATTERNS

Reap zombies non-blocking (in SIGCHLD handler)

```
while(pid==waitpid(-1,&st,WNOHANG)>0)
    /* update job table */;
```

Daemonize

```
if(fork()) exit(0); // parent
setsid();
if(fork()) exit(0); // session leader
chdir("/"); umask(0);
close(0); close(1); close(2);
```

Redirect stdout to file

```
int fd=open("out.txt",
            O_WRONLY|O_CREAT|O_TRUNC,0644);
dup2(fd,STDOUT_FILENO); close(fd);
execvp(argv[0],argv);
```

Self-pipe (sig-safe IPC)

```
int sp[2]; pipe(sp);
void handler(int s){ write(sp[1],"x",1); }
// main: select/read sp[0]
```

Mutex+CV producer/consumer

```
// Producer:
pthread_mutex_lock(&m);
while(full) pthread_cond_wait(&not_full,&m);
buf[incr++]=v; count++;
pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&m);
// Consumer:
pthread_mutex_lock(&m);
while(empty) pthread_cond_wait(&not_empty,&m);
v=buf[out++]; count--;
pthread_cond_signal(&not_full);
pthread_mutex_unlock(&m);
```

Read EXACTLY n (handle EINTR/partial)

```
ssize_t readn(int fd,void*b,size_t n){
    size_t left=n; char*p=b; ssize_t r;
    while(left>0){
        if((r=read(fd,p,left))<0){
            if(errno==EINTR) continue;
            return -1;
        }
        if(r==0) break;
        left-=r; p+=r;
    }
    return n-left;
}
```

Fix Akash's buggy code (Signals Part B)

```
// CHANGE sigprocmask:
sigprocmask(SIG_BLOCK,&block_mask,NULL);
// CHANGE handler from SIG_IGN to real:
void nop(int s){}
sa.sa_handler = nop;
// Now sigsuspend can wake.
```

Atomic counter w/ mutex

```
pthread_mutex_lock(&m);
counter++;
pthread_mutex_unlock(&m);
```

Race-free SIGCHLD wait loop

```
sigset_t b,o; sigemptyset(&b);
sigaddset(&b,SIGCHLD);
sigprocmask(SIG_BLOCK,&b,&o);
while(job_running){
    sigset_t e; sigemptyset(&e);
    sigsuspend(&e);
    /* reap in handler or here w/ WNOHANG */
}
```

Read whole file

```
int fd=open(path,O_RDONLY);
struct stat st; fstat(fd,&st);
char *buf=malloc(st.st_size);
readn(fd,buf,st.st_size);
close(fd);
```

GOTCHAS — DO NOT FORGET

- **sigsuspend(&mask)**; **mask** = sigs to BLOCK. Empty set = unblock everything.
- **SIG_IGN discards** signal. sigsuspend can't wake on ignored sig.
- **SIGKILL & SIGSTOP** uncatchable/unblockable/unignorable.
- After exec: FDs persist, handlers reset to DFL, **SIG_IGN persists**.
- Pipe EOF only when ALL write ends closed (including parent's).
- dup2(o,n) does NOT close o. Close after.
- WNOHANG: 0 if child alive no-change; -1 (ECHILD) no children.
- read returns **0 on EOF**, not -1.
- Write to no-ready pipe → SIGPIPE (default terminates!). Handle or ignore.
- Forked child inherits stdio buffers → fflush before fork.
- Standard signals don't queue (1 pending max).
- Handler functions: only async-signal-safe (write OK, printf/malloc NO).
- volatile sig_atomic_t for vars touched in handlers.
- Threads share heap → use locks.
- **errno is thread-local**.
- pthread_cond_wait **MUST** be in while loop (spurious wakeups).
- Don't return ptr to local from thread fn.
- Page size = 2ⁿ (offset bits). ≠ PTE size.
- TLB caches PTEs, not data.
- SRTF needs estimates — impractical → real systems use CFS/MLFQ.
- RR equal opp ≠ equal CPU time.
- Lock-everything alone doesn't prevent deadlock — order matters too.
- External frag = scattered holes; internal = bytes wasted inside block.
- Hard link count in **inode**, not dirent.
- Inode bitmap = inodes; block bitmap = data blocks.
- fork is COW; child VAS virtual until write.
- mmap returns MAP_FAILED ((void*)-1), not NULL.
- Belady's: FIFO can have more faults with more frames.
- Atomic pipe write ≤ PIPE_BUF (4096).
- read can return less than asked — always loop.
- Filenames in dirents, not inodes.
- SIGPIPE default = term — easy to forget piping.
- Sigsuspend always returns -1 errno=EINTR.
- printf before fork → both buffer → "hihi" on exit.
- Cond var without while = bug.

SCHEDULING DIAGRAM TIPS

- Track remaining time per task at each step.
- Ties (SRTF): usually keep current task (avoid switch).
- Preemptive: on every arrival, recompare.
- RR: explicit ready queue; preempt at quantum end; arrivals to BACK.
- Recurring D*: mark run, then "off" for k slots.
- Avg wait = (sum of start – arrival) / N.
- Turnaround = completion – arrival.
- Response = first run – arrival.
- For CFS: pick min vruntime each tick; vruntime += time_used × weight.

FAT vs Inode summary

	FAT	Inode
Free space	FAT entries = 0	Block bitmap
File meta	In dirent	In inode
Random access	Walk chain — slow	Index — fast
Hard links	Hard	Natural (refcount)
Used	DOS, USB	Linux/Unix

POWERS OF 2 — QUICK

2 ⁿ	Value
2 ¹⁰	1 KB = 1024
2 ¹²	4 KB (page)
2 ¹⁶	64 KB
2 ¹⁸	256 KB
2 ²⁰	1 MB
2 ²¹	2 MB (huge pg)
2 ³⁰	1 GB
2 ³²	4 GB
2 ⁴⁰	1 TB
2 ⁴⁸	x86-64 VA range

EXAM TACTICS

- **Read WHOLE exam first**. Bank easy points.
- "Explain WHY" → mention **mechanism** (kernel does X, hw does Y). Don't restate symptom.
- "Specific hardware structure" → **TLB, cache line, page-table walker, MMU, L1/L2 cache**. NOT "memory".
- Coding: skip includes/error checking unless asked. Get structure right.
- MCQ: one BEST answer, no select-all tricks.
- Cumulative — drill processes/signals from early term.
- Defaults: SIGINT/TERM/PIPE/ALRM/HUP = term; SIGCHLD = ignore; SIGTSTP/TTIN/TTOU = stop; SIGCONT = resume; SIGSEGV/QUIT/BUS/FPE = core.
- 2 hr — ~1 pt/min. Don't rush, don't linger.
- Show work on scheduling — partial credit.
- Don't know? Write SOMETHING in right vocab — partial credit.
- Initial each page (separation risk).
- Bring: pencil, eraser, 2 cheat sheets, water. "Before" vs "after" matters in signals.